

Model learning

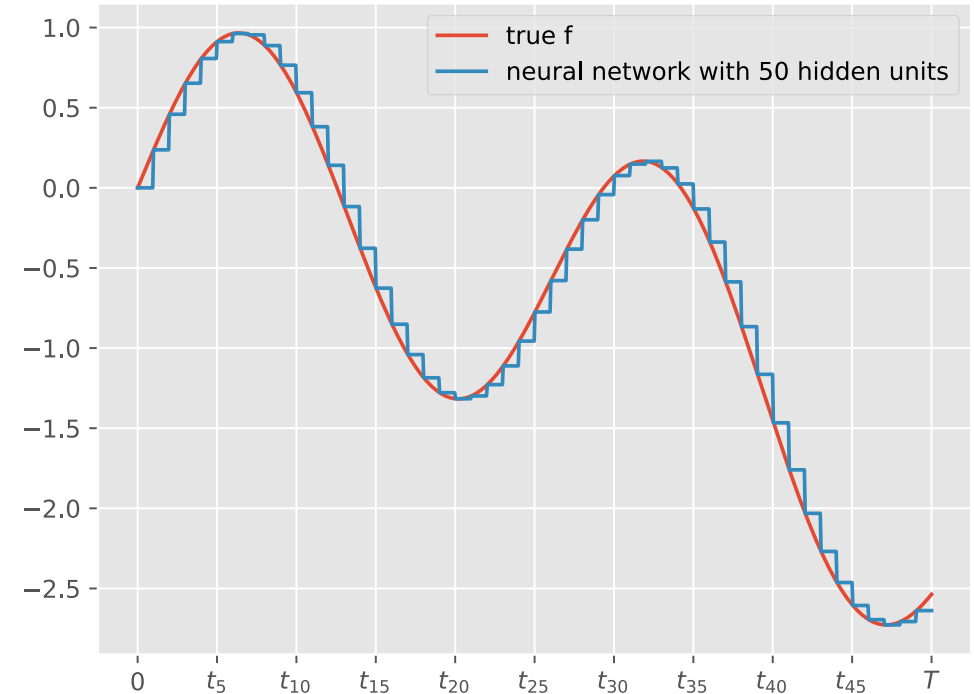
Matt Engelhard

On homework 1

- Goals:
 - Touch MIMIC-IV data
 - Begin to learn PyTorch
 - Explore training dynamics
 - Conduct careful model evaluation
- You may need to bend the instructions a bit in some cases
 - Hidden layer sizes
 - Regularization
- Low bar to reach out with questions or uncertainties

On ReLU

- Networks with sigmoid activations are surprisingly inconsistent
 - Fixed scale
 - Vanishing gradient
 - Piecewise constant approximations
- Why ReLU?
 - Variable scale
 - Non-trivial gradient over a larger range
 - Piecewise linear approximations
 - **Much more effective and consistent in practice**
- Many effective variants on the ReLU



Consider: how many ReLU hidden units needed to get an equally faithful approximation?

Report back: activation functions

Why are non-linear activation functions so important?

Would it ever make sense to build a neural network with linear activation functions (or without activation functions)?

Why or why not?

Overview of learning

How does training, or learning, work?

I want to find the *best* model parameters.

First, I need to define what *best* means.

- > for binary classification, we use the *cross-entropy loss*

- > the *best* parameters are those that minimize it

Then, I need a strategy for adjusting my parameters to make my model *better*...

- > our strategy for reducing it is called *stochastic gradient descent*

- > this is an example of an *optimization algorithm*

The **simplicity** of this recipe is a key strength of ML/AI

- > simple learning routines, complex models

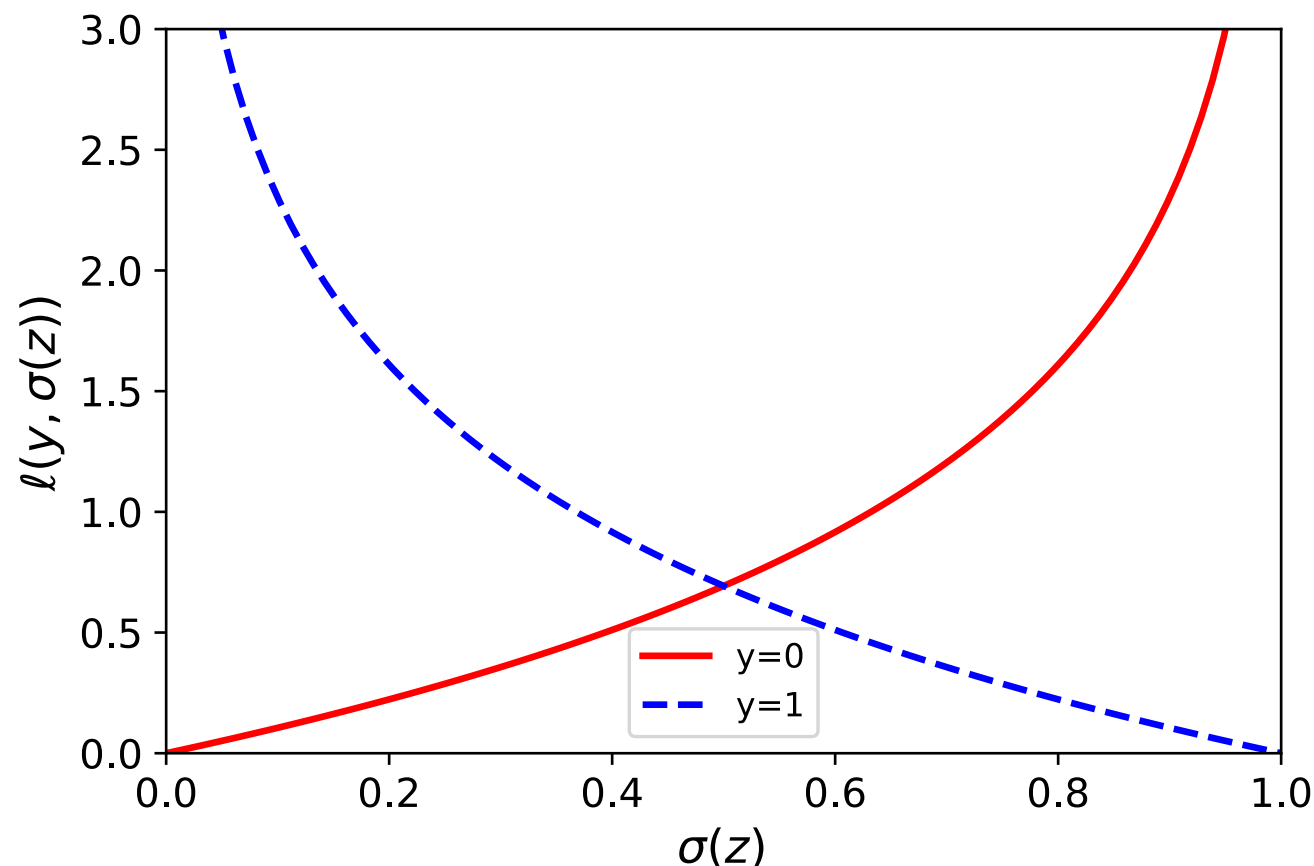
- > PyTorch and other ML libraries are designed to make this easy

Loss Function for Binary Classification

“cross-entropy loss”

When the outcome y is 0,
the model-predicted
probability p must be close
to 0, otherwise the loss
(i.e., the penalty) is large

When the outcome y is 1,
the model-predicted
probability p must be close
to 1, otherwise the loss
(i.e., the penalty) is large



A Simple Strategy for Optimizing Parameters

Here's our game.

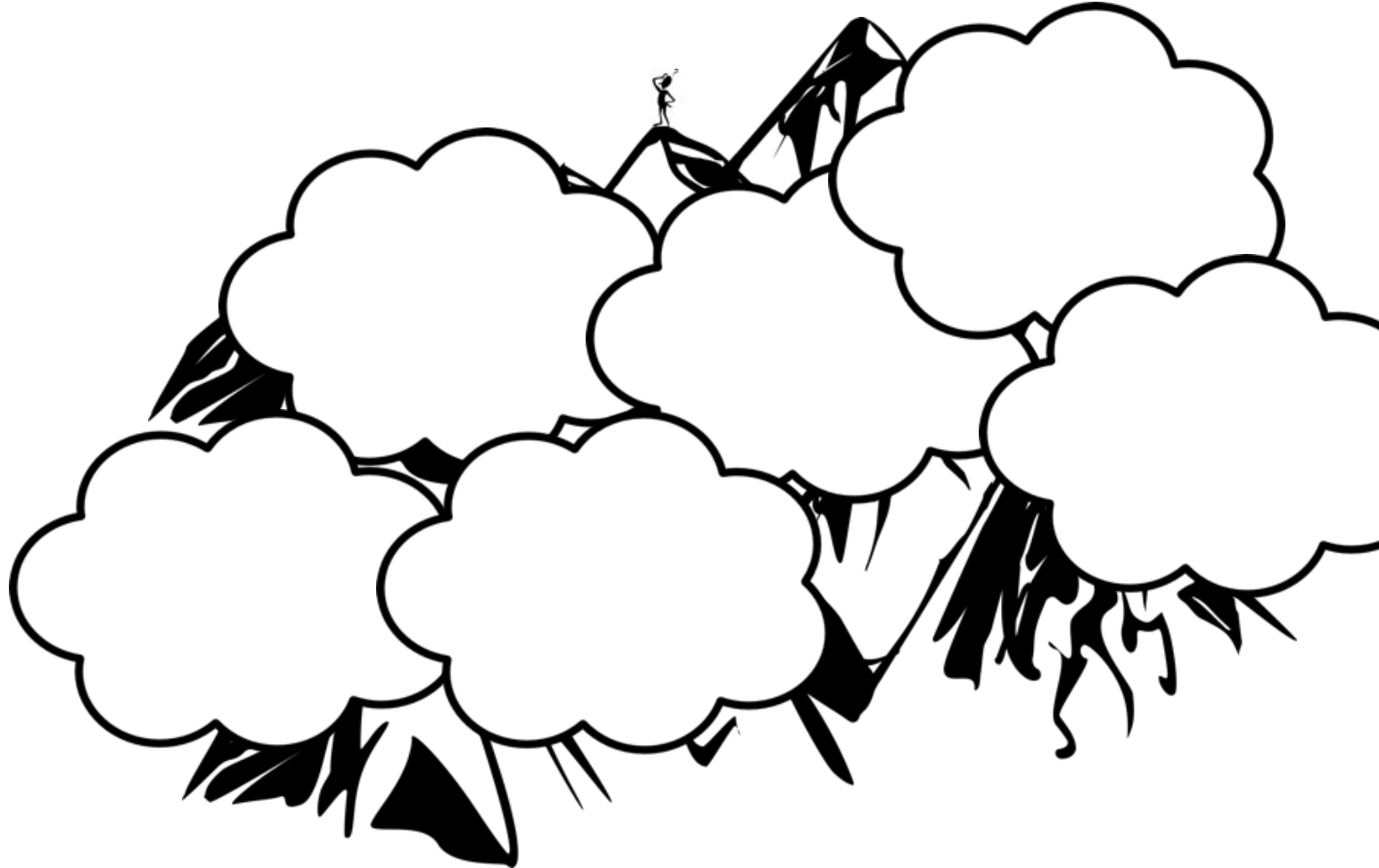
1. We want to reduce the loss as much as we can by adjusting our model's parameters.
2. We can move any parameter up or down from its current value.
3. If the loss increases, *we're going the wrong way*, and we should try moving that same parameter in the opposite direction.
4. If the loss decreases, *we're going the right way*, and we should keep moving that parameter in that same direction until the loss stops decreasing.
5. We will keep trying this for all of the parameters until we can no longer decrease the loss; then we're done.

Leter we'll try this strategy on a small dataset.

MORTALITY PREDICTION WORKSHEET													
COVARIATES						OUTCOMES AND PREDICTIONS							
patient	age	age_normalized	female	temp	temp_normalized	mortality	predicted_log_odds	predicted_prob	prediction	correct?	loss		
0	30.5	-0.5	0	105.0	2.4	1	0.00	0.50	0	0	0.3010		
1	74.0	1.1	1	96.7	-0.8	0	0.00	0.50	0	1	0.3010		
2	27.4	-0.6	0	96.1	-1.0	0	0.00	0.50	0	1	0.3010		
3	0.1	-1.5	1	98.5	-0.1	0	0.00	0.50	0	1	0.3010		
4	0.7	-1.5	1	96.5	-0.9	0	0.00	0.50	0	1	0.3010		
5	49.9	0.2	1	97.1	-0.6	0	0.00	0.50	0	1	0.3010		
6	72.9	1.0	1	100.1	0.5	1	0.00	0.50	0	0	0.3010		
7	29.1	-0.5	1	99.6	0.3	0	0.00	0.50	0	1	0.3010		
8	83.5	1.4	1	100.6	0.7	1	0.00	0.50	0	0	0.3010		
9	82.3	1.4	1	95.2	-1.3	1	0.00	0.50	0	0	0.3010		
10	23.7	-0.7	0	99.4	0.2	1	0.00	0.50	0	0	0.3010		
11	12.9	-1.1	0	96.6	-0.8	0	0.00	0.50	0	1	0.3010		
12	53.9	0.4	1	100.3	0.6	0	0.00	0.50	0	1	0.3010		
13	18.8	-0.9	0	98.6	0.0	0	0.00	0.50	0	1	0.3010		
14	51.8	0.3	0	98.5	-0.1	0	0.00	0.50	0	1	0.3010		
15	3.3	-1.4	0	94.6	-1.6	0	0.00	0.50	0	1	0.3010		
16	69.7	0.9	0	99.1	0.1	0	0.00	0.50	0	1	0.3010		
17	60.4	0.6	1	104.2	2.1	1	0.00	0.50	0	0	0.3010		
18	73.6	1.1	1	99.1	0.1	1	0.00	0.50	0	0	0.3010		
19	53.3	0.3	1	99.1	0.1	0	0.00	0.50	0	1	0.3010		
PARAMETERS						PERFORMANCE							
	guess	0.00	0.00		0.00					accuracy	avg_loss		
	optimal									0.65	0.3010		

Parameter Optimization by Gradient Descent

- We are trying to get to the lowest point (minimum loss).
- We do not know where it is (cannot brute force evaluate all parameter combinations).
- We cannot see clearly in any direction (loss is a complex, high-dimensional function of parameters).
- So, we test the ground around us and walk downhill (calculate gradient, step in opposite direction).



Defining our loss

Likelihood from first principles

- Our model predicts the *probability* of death for each patient.
- If we change the parameters, we change the predicted probabilities for each patient.

$y_1 = 1$
(dies)



$p_1?$

x

$y_2 = 0$
(survives)



$p_2?$

x

$y_3 = 0$
(survives)



$p_3?$

Likelihood from first principles

- Suppose we predict:

- $p_1 = .1$
- $p_2 = .9$
- $p_3 = .7$

- Is this a good model?
Why or why not?

$y_1 = 1$
(dies)



$p_1?$

x

$y_2 = 0$
(survives)



$p_2?$

x

$y_3 = 0$
(survives)



$p_3?$

Likelihood from first principles

- Suppose we predict:
 - $p_1 = .8$
 - $p_2 = .3$
 - $p_3 = .1$
- Is this a good model?
Why or why not?

$y_1 = 1$
(dies)



$p_1?$

x

$y_2 = 0$
(survives)



$p_2?$

x

$y_3 = 0$
(survives)



$p_3?$

Likelihood from first principles

- Suppose we predict:
 - $p_1 = .8$
 - $p_2 = .3$
 - $p_3 = .1$
- What is the probability of the observed outcomes?
- This is called the likelihood. We want to find parameters that maximize it.

$y_1 = 1$
(dies)



$p_1?$

x

$y_2 = 0$
(survives)



$p_2?$

x

$y_3 = 0$
(survives)



$p_3?$

Likelihood from first principles

- Suppose we predict:
 - $p_1 = .8$
 - $p_2 = .3$
 - $p_3 = .1$
- Is this a good model?
Why or why not?
- Our parameters affect *all* the predictions: changing a parameter to *decrease* y_2 may also *increase* y_3

$y_1 = 1$
(dies)



$p_1?$

$y_2 = 0$
(survives)



$p_2?$

$y_3 = 0$
(survives)



$p_3?$

x

x

Likelihood from first principles

- Suppose we predict:
 - $p_1 = .99$
 - $p_2 = .01$
 - $p_3 = .01$
- Is this a good model?
Why or why not?
- Our parameters affect *all* the predictions: changing a parameter to *decrease* y_2 may also *increase* y_3

$y_1 = 1$
(dies)



$p_1?$

$y_2 = 0$
(survives)



$p_2?$

$y_3 = 0$
(survives)



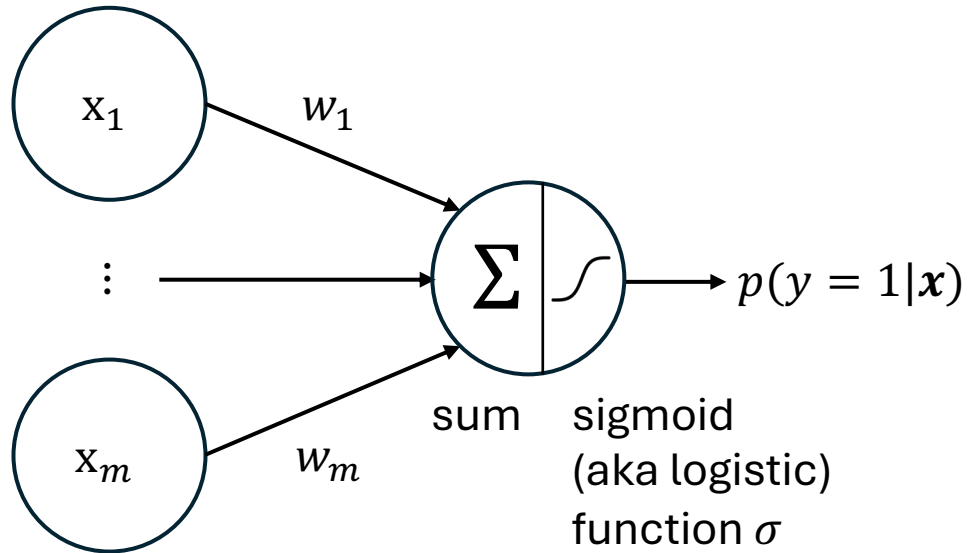
$p_3?$

x

x

Likelihood from first principles

Probability of all the events $y_1, \dots, y_N$ given our current model parameters



The predicted probability of y_i is:

$$\begin{cases} \sigma(z_i) & \text{if } y_i = 1 \\ 1 - \sigma(z_i) & \text{if } y_i = 0 \end{cases}$$

Written more compactly:

$$\sigma(z_i)^{y_i} (1 - \sigma(z_i))^{1-y_i}$$

Multiply over all observations:

$$\prod_{i=1}^N \sigma(z_i)^{y_i} (1 - \sigma(z_i))^{1-y_i}$$

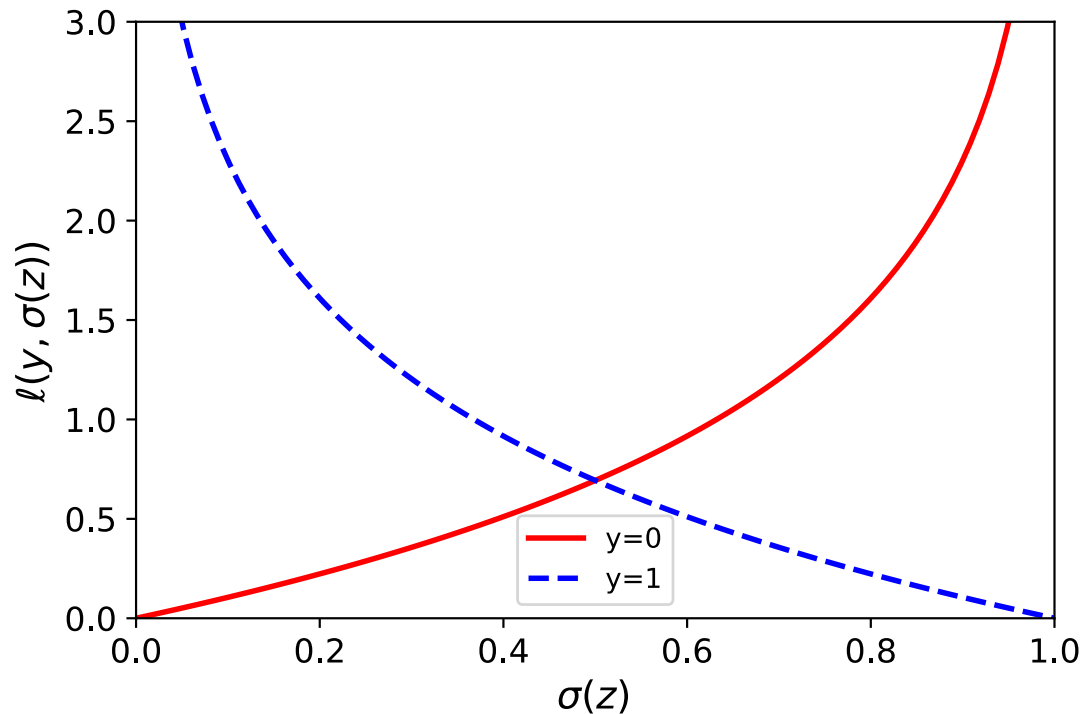
For numerical stability, use the *log-likelihood*:

$$\sum_{i=1}^N y_i \log \sigma(z_i) + (1 - y_i) \log(1 - \sigma(z_i))$$

By convention, minimize the negative log-likelihood

Likelihood from first principles

Probability of all the events $y_1, \dots, y_N$ given our current model parameters



“cross-entropy loss” or “log loss”

$$y_i \log \sigma(z_i) + (1 - y_i) \log(1 - \sigma(z_i))$$

The predicted probability of y_i is:

$$\begin{cases} \sigma(z_i) & \text{if } y_i = 1 \\ 1 - \sigma(z_i) & \text{if } y_i = 0 \end{cases}$$

Written more compactly:

$$\sigma(z_i)^{y_i} (1 - \sigma(z_i))^{1-y_i}$$

Multiply over all observations:

$$\prod_{i=1}^N \sigma(z_i)^{y_i} (1 - \sigma(z_i))^{1-y_i}$$

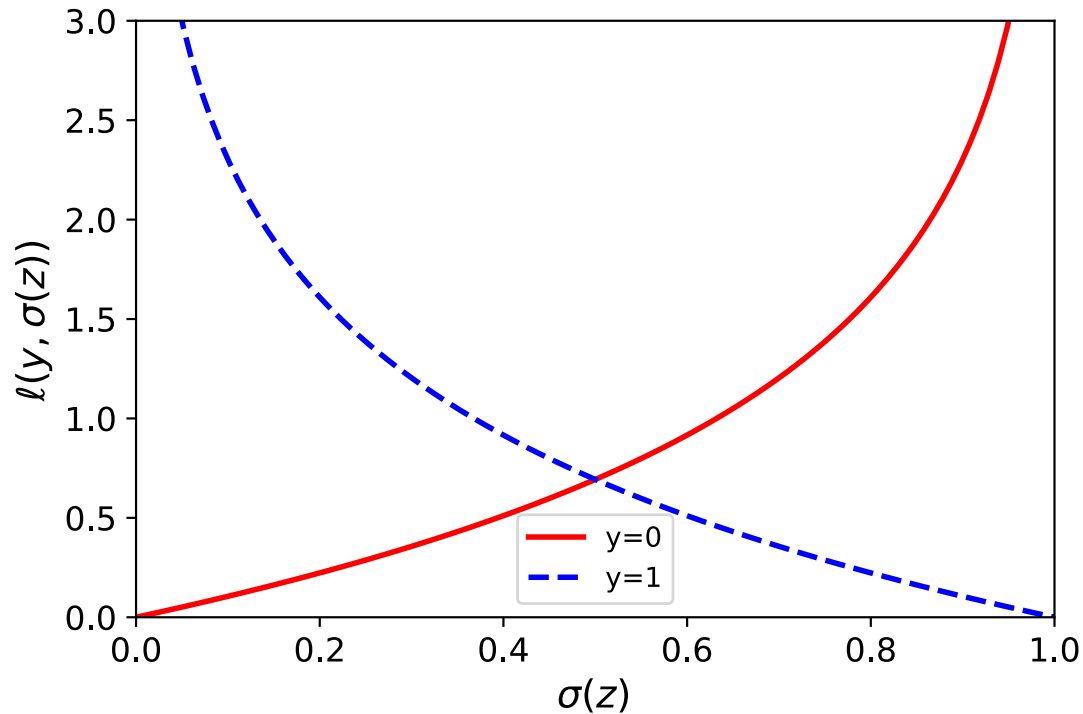
For numerical stability, use the *log-likelihood*:

$$\sum_{i=1}^N y_i \log \sigma(z_i) + (1 - y_i) \log(1 - \sigma(z_i))$$

By convention, minimize the negative log-likelihood

Likelihood from first principles

Probability of all the events $y_1, \dots, y_N$ given our current model parameters



Notes on this loss:

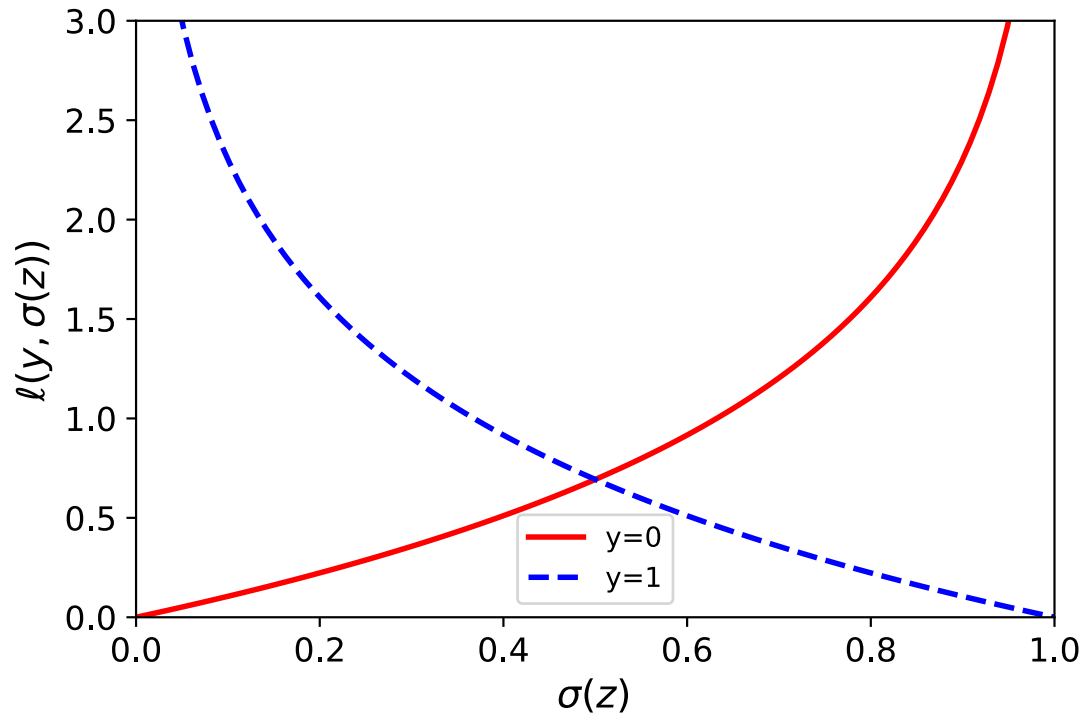
- Not just for logistic regression

“cross-entropy loss” or “log loss”

$$y_i \log \sigma(z_i) + (1 - y_i) \log(1 - \sigma(z_i))$$

Likelihood from first principles

Probability of all the events $y_1, \dots, y_N$ given our current model parameters



Notes on this loss:

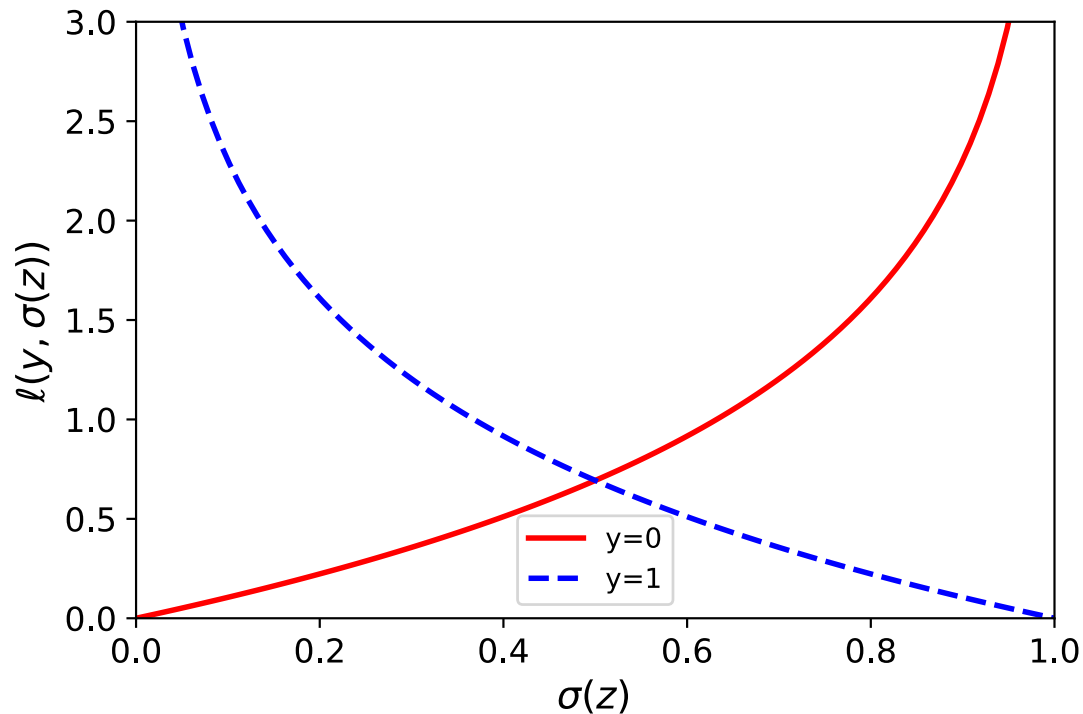
- Not just for logistic regression
- Consider the loss for confident incorrect predictions

“cross-entropy loss” or “log loss”

$$y_i \log \sigma(z_i) + (1 - y_i) \log(1 - \sigma(z_i))$$

Likelihood from first principles

Probability of all the events $y_1, \dots, y_N$ given our current model parameters

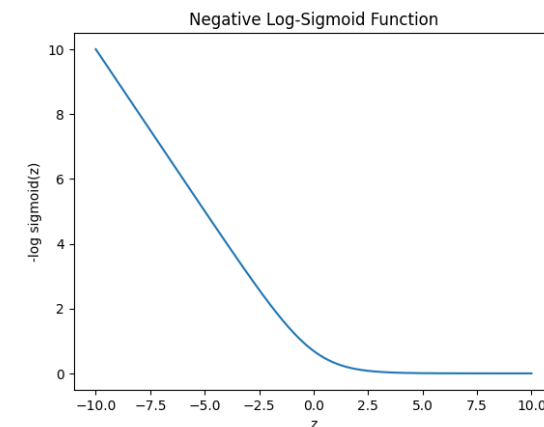


“cross-entropy loss” or “log loss”

$$y_i \log \sigma(z_i) + (1 - y_i) \log(1 - \sigma(z_i))$$

Notes on this loss:

- Not just for logistic regression
- Consider the loss for confident incorrect predictions
- Note that $-\log \sigma(z)$ is strictly convex



Report back: choice of loss function

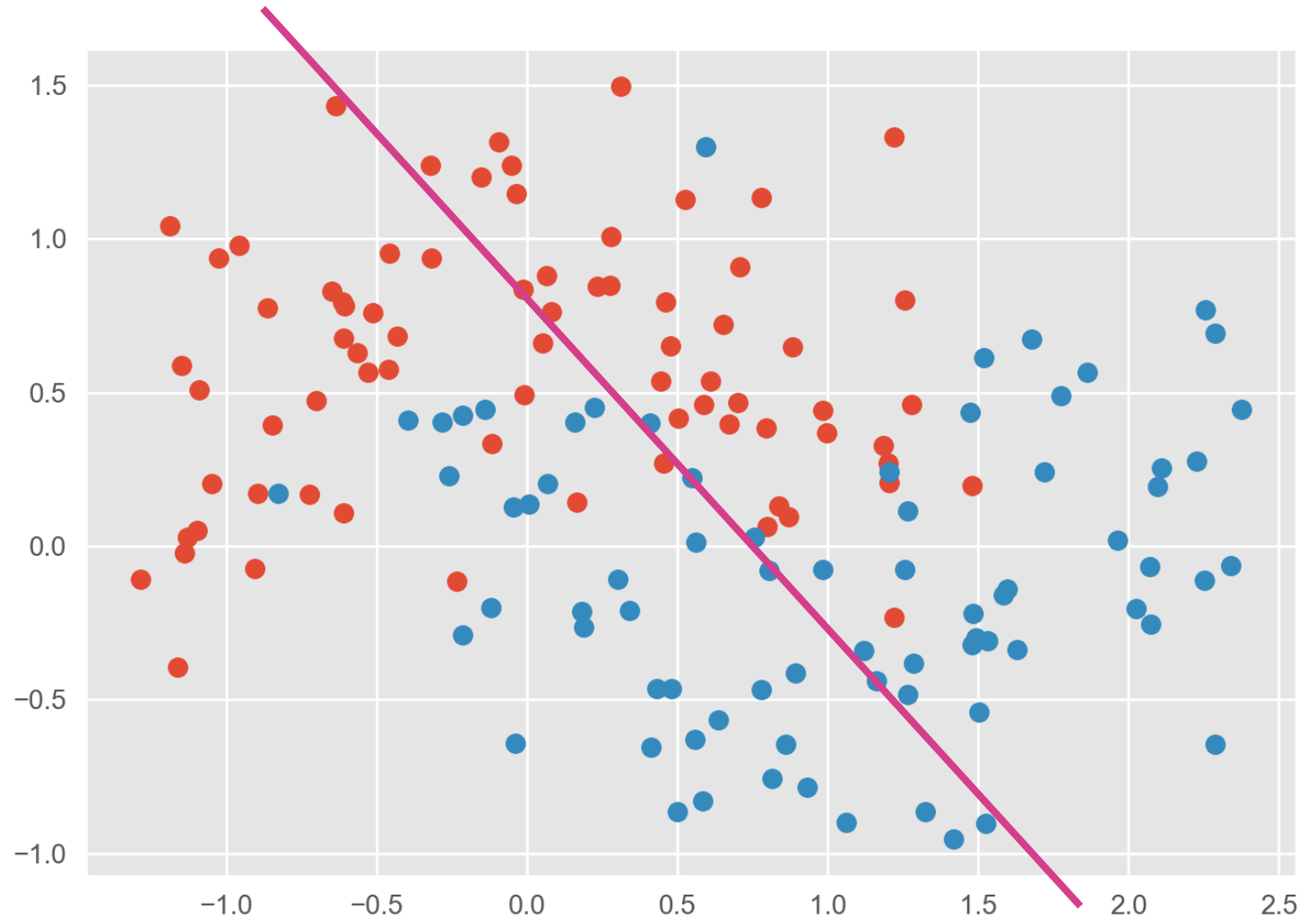
Often we evaluate performance using the area under the ROC curve (AUC). Would it make sense to directly optimize this by using $-AUC$ as our loss function? Why or why not?

An initial look at regularization

Let's teach a neural network to identify cancer

Initial decision boundary

(epoch 0)

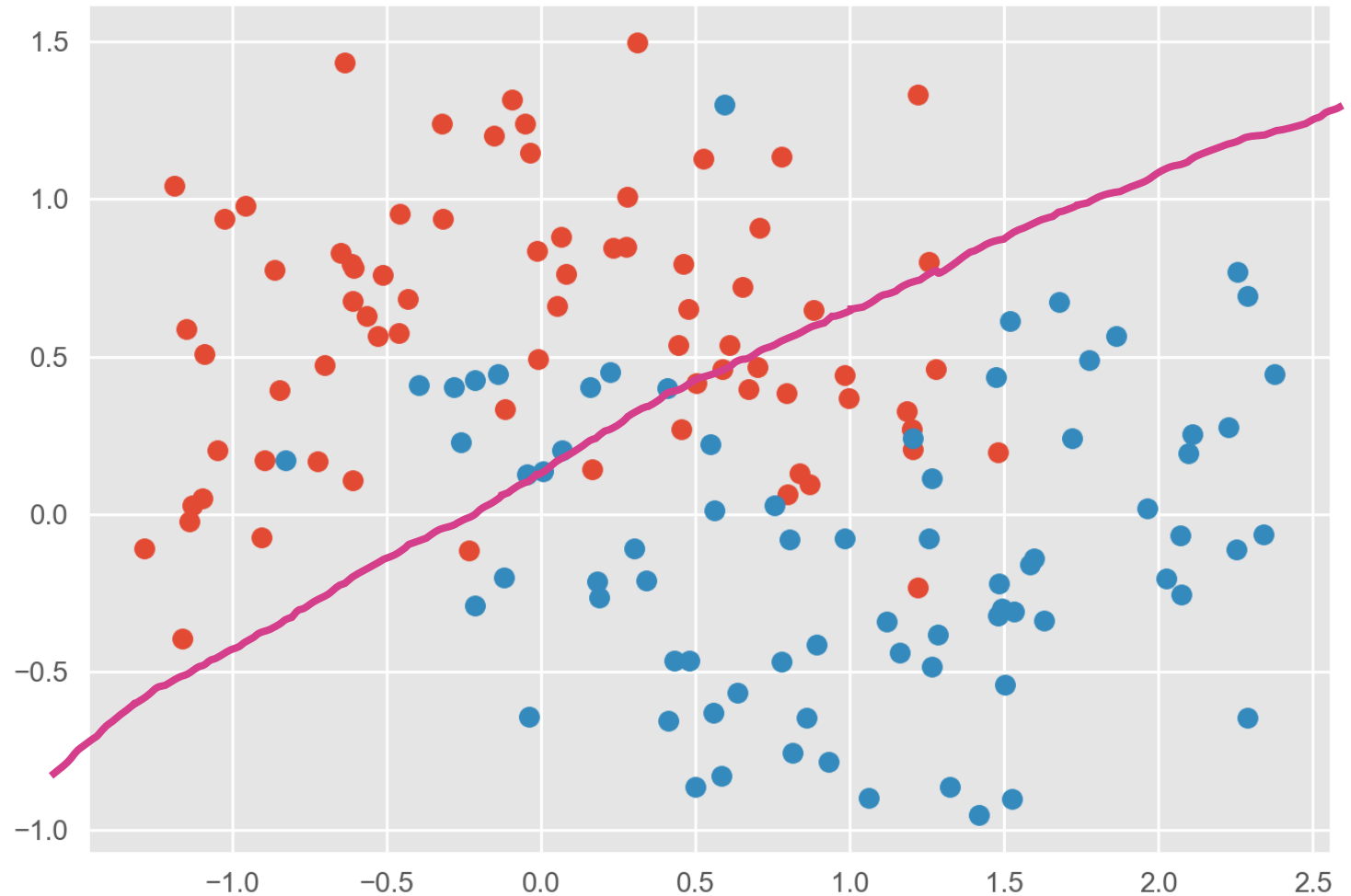


Let's teach a neural network to identify cancer

Updated decision boundary

(epoch 1)

Sure.

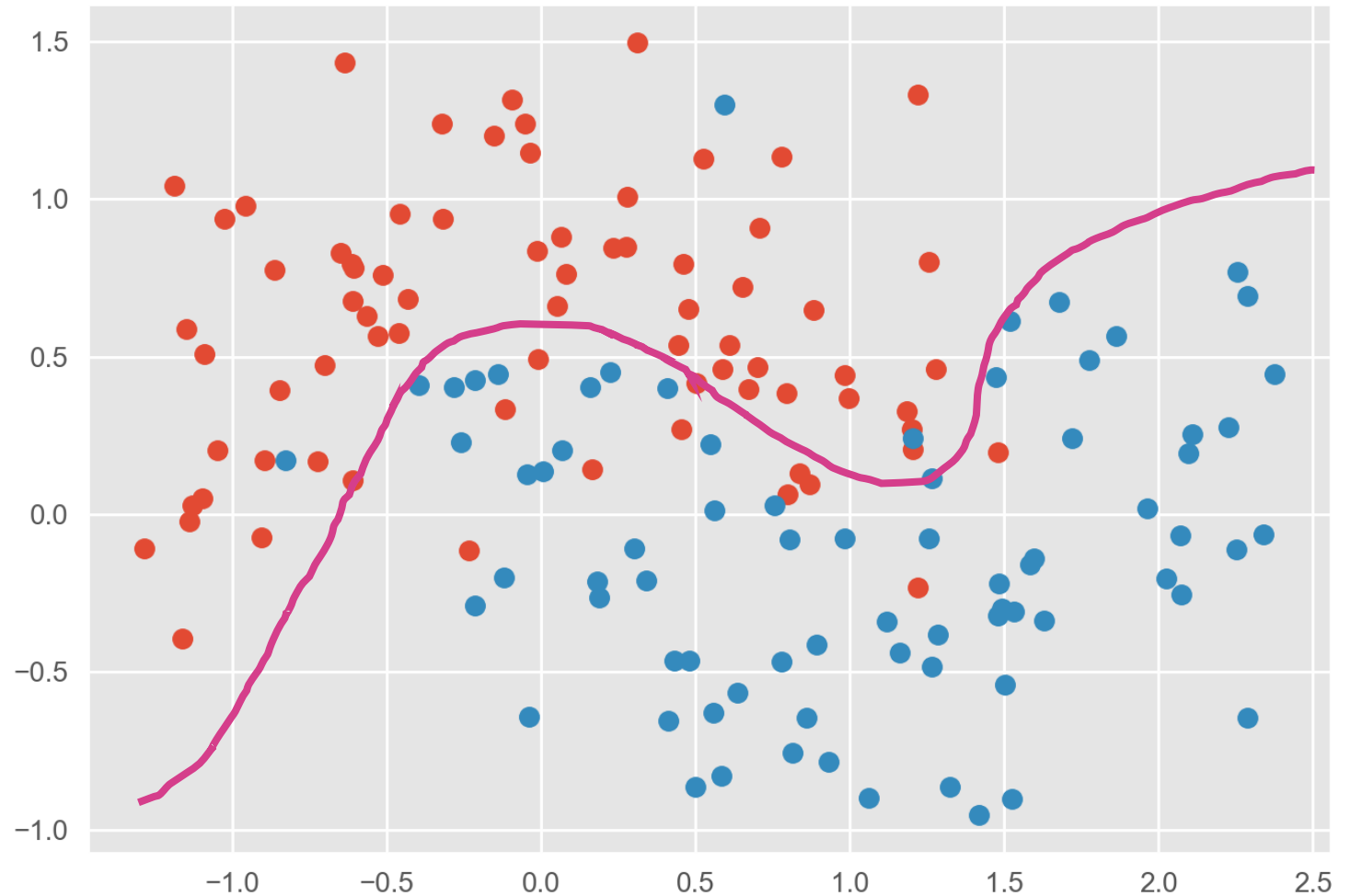


Let's teach a neural network to identify cancer

Updated decision boundary

(epoch 4)

Wow!



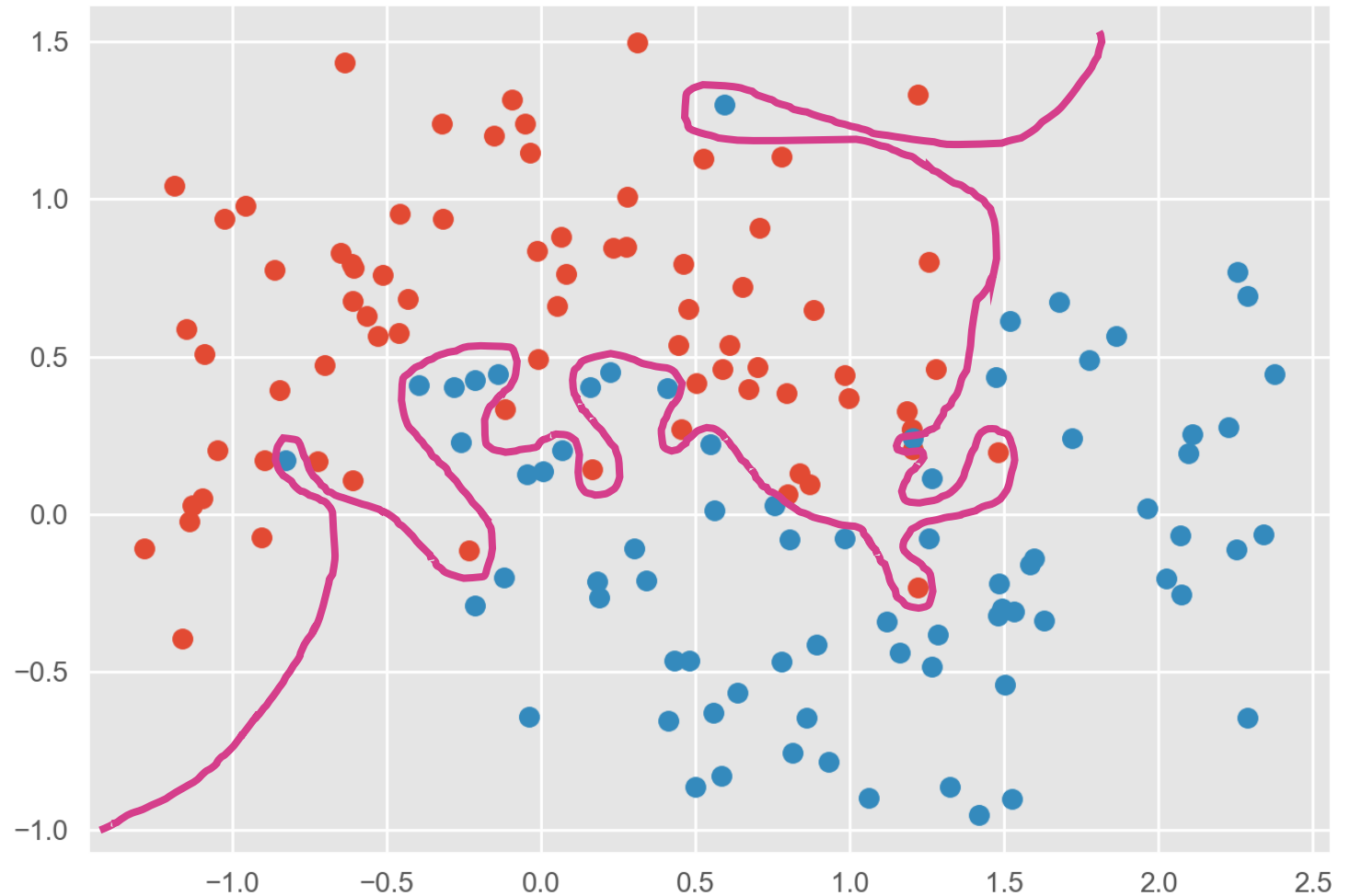
Let's teach a neural network to identify cancer

Updated decision boundary

(epoch 23)

Hmm...

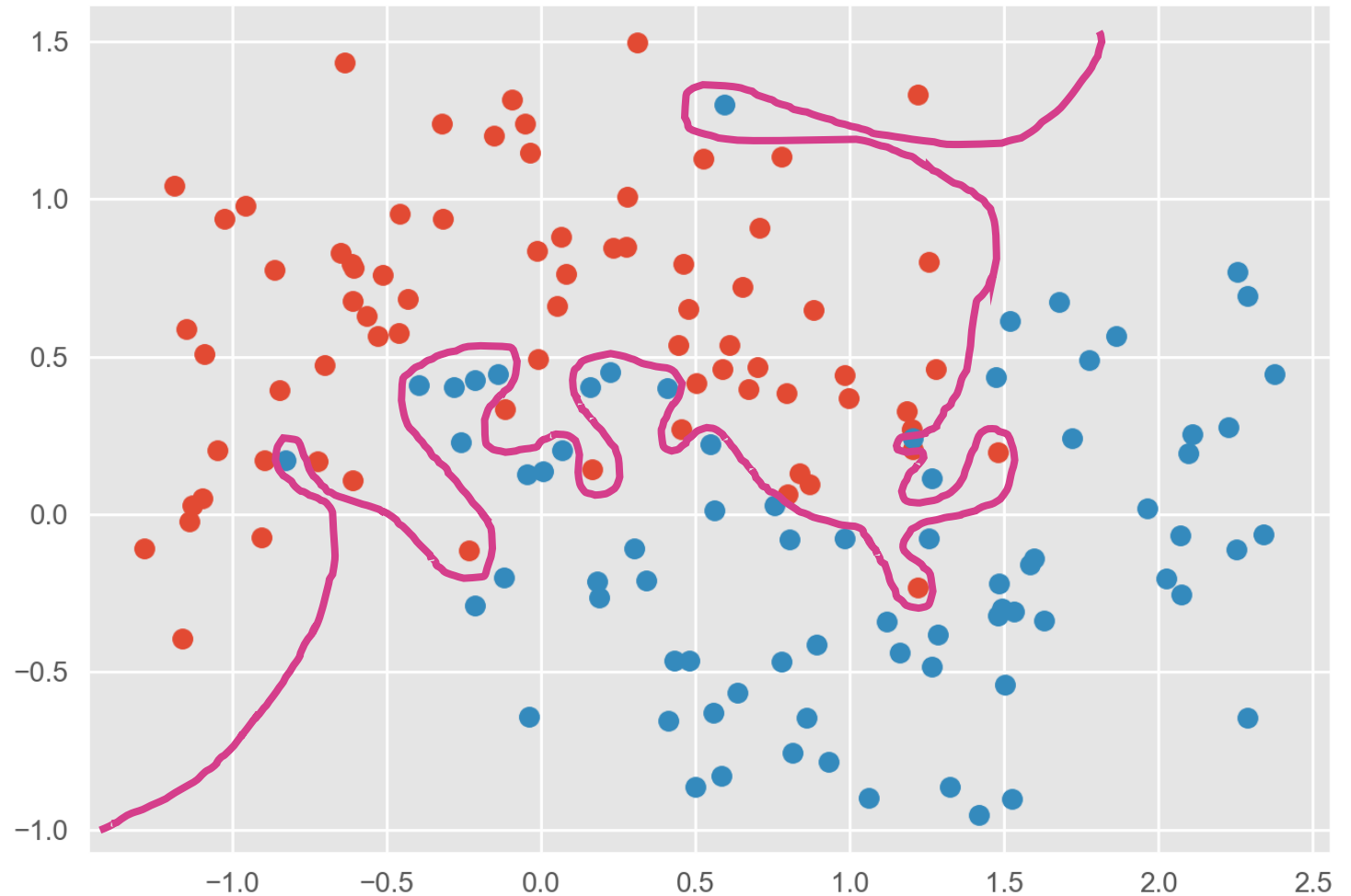
It doesn't know when to stop.



We need a strategy to decide when to stop.

Strategy 1:
Penalize complexity
(regularization)

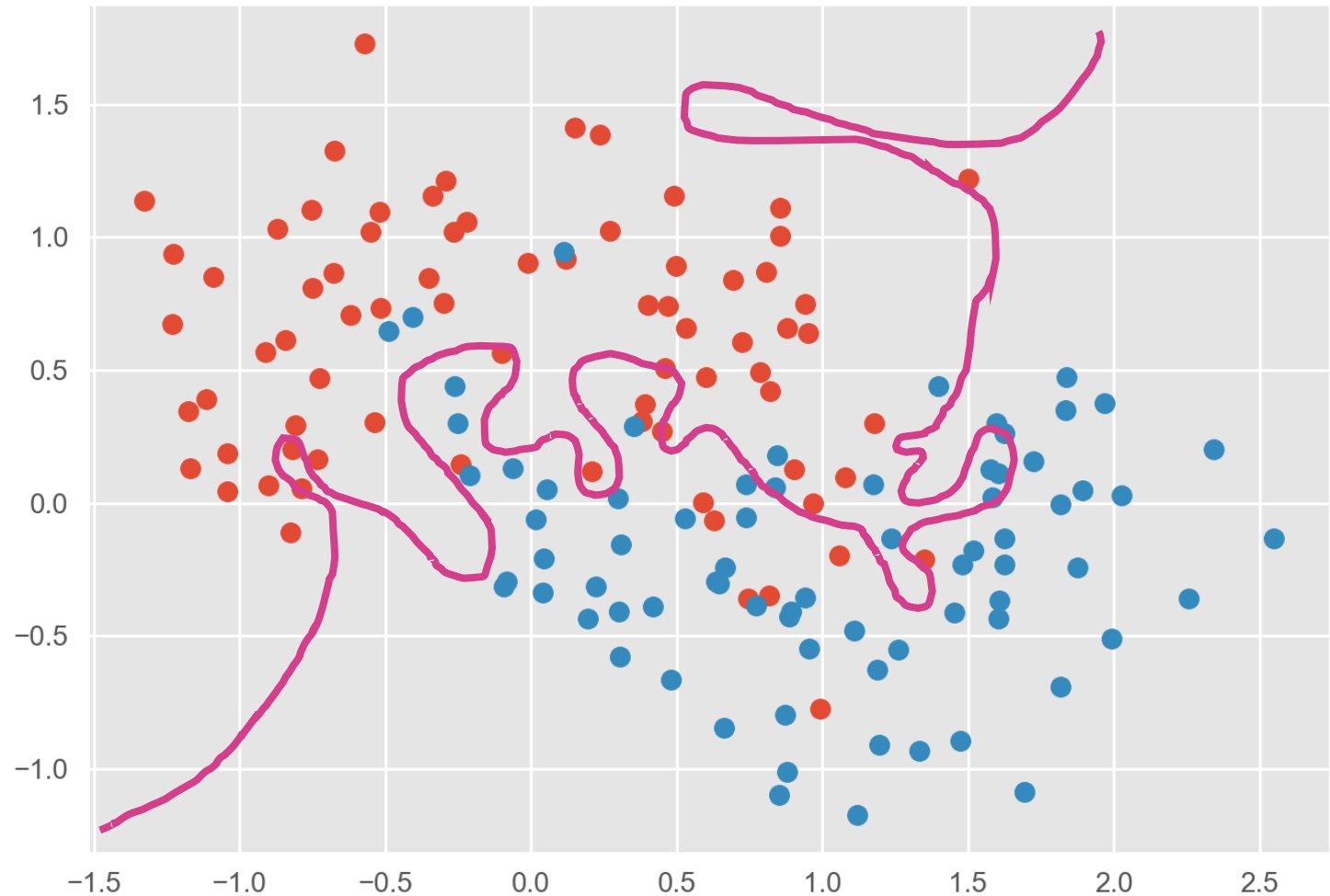
We could quantify how curvy
the line is and add that value
to the loss.



We need a strategy to decide when to stop.

Strategy 2:
*Keep checking the validation
set*
(early stopping)

When our boundary no longer
works well on new data, we
know we've gone too far.



Strategy 1: Explicit Regularization

- We add a term to the loss (L) that quantifies complexity
- By minimizing the loss, we're now striking a balance between two competing objectives:
 1. Improve goodness of fit (i.e. reduce cross-entropy loss H)
 2. Limit complexity (i.e. reduce regularization term R)
- The hyperparameter λ controls how much we care about (1) vs (2)

$$L = H + \lambda R$$

Examples of Explicit Regularization

- L_2 (i.e. *ridge*) regularization: $R = \|\theta\|_2^2$
- In other words, we penalize the sum of squares of parameters
- Common in linear models and in neural networks

$$L = H + \lambda R$$

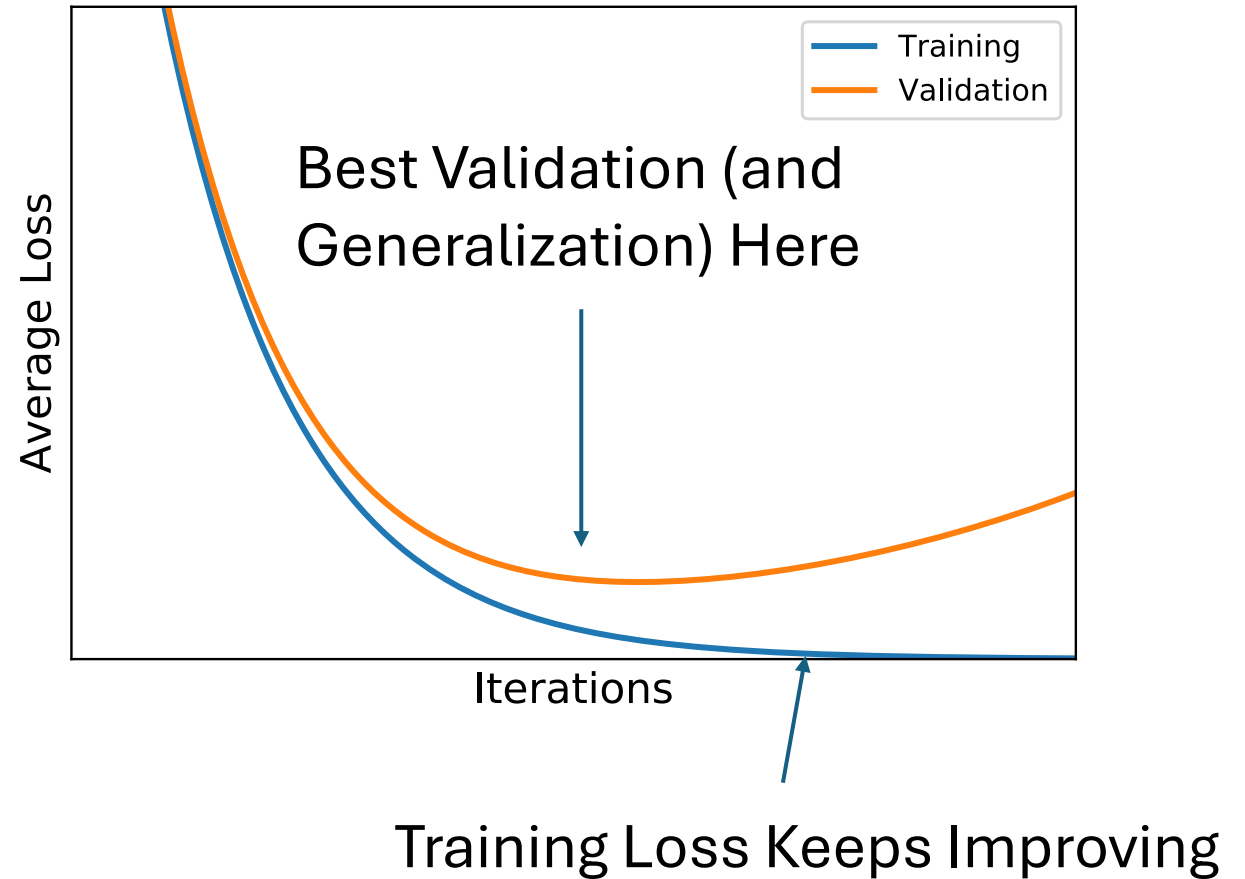
Examples of Explicit Regularization

- L_1 (i.e. *LASSO*) regularization: $R = \|\theta\|_1$
- In other words, we penalize the sum of absolute values of parameters
- Common in linear models; less so in neural networks
- Tends to shrink many parameters to 0
 - This may be viewed as a form of feature selection
 - This also improves interpretability

$$L = H + \lambda R$$

Strategy 2: Early Stopping

- During optimization, we can check the validation loss as we go.
- Instead of optimizing to convergence, we can optimize until the *validation* loss stops improving
 - Saves computational cost
 - Performs better on validation (and test) sets
- Widely used technique



Strategy 3: Dropout

- Exclusive to neural networks
- During training, we set some hidden features to zero at random (different features for each training step)
- This has a dual effect:
 - Parameter shrinkage (similar to L_2)
 - Encourages redundancy / redundant features; in other words, the network must model the relationship between x and y in multiple ways

Mitigating Overfitting in Practice

- Machine learning almost always uses some form of regularization and/or early stopping
- Very common to use multiple forms at once
 - Dropout + early stopping in neural networks
 - Combination of L1 and L2 regularization in linear models (i.e. elastic net)
- The validation set is the key: used to choose between strategies
- A key, often time-consuming step in ML/AI research projects.

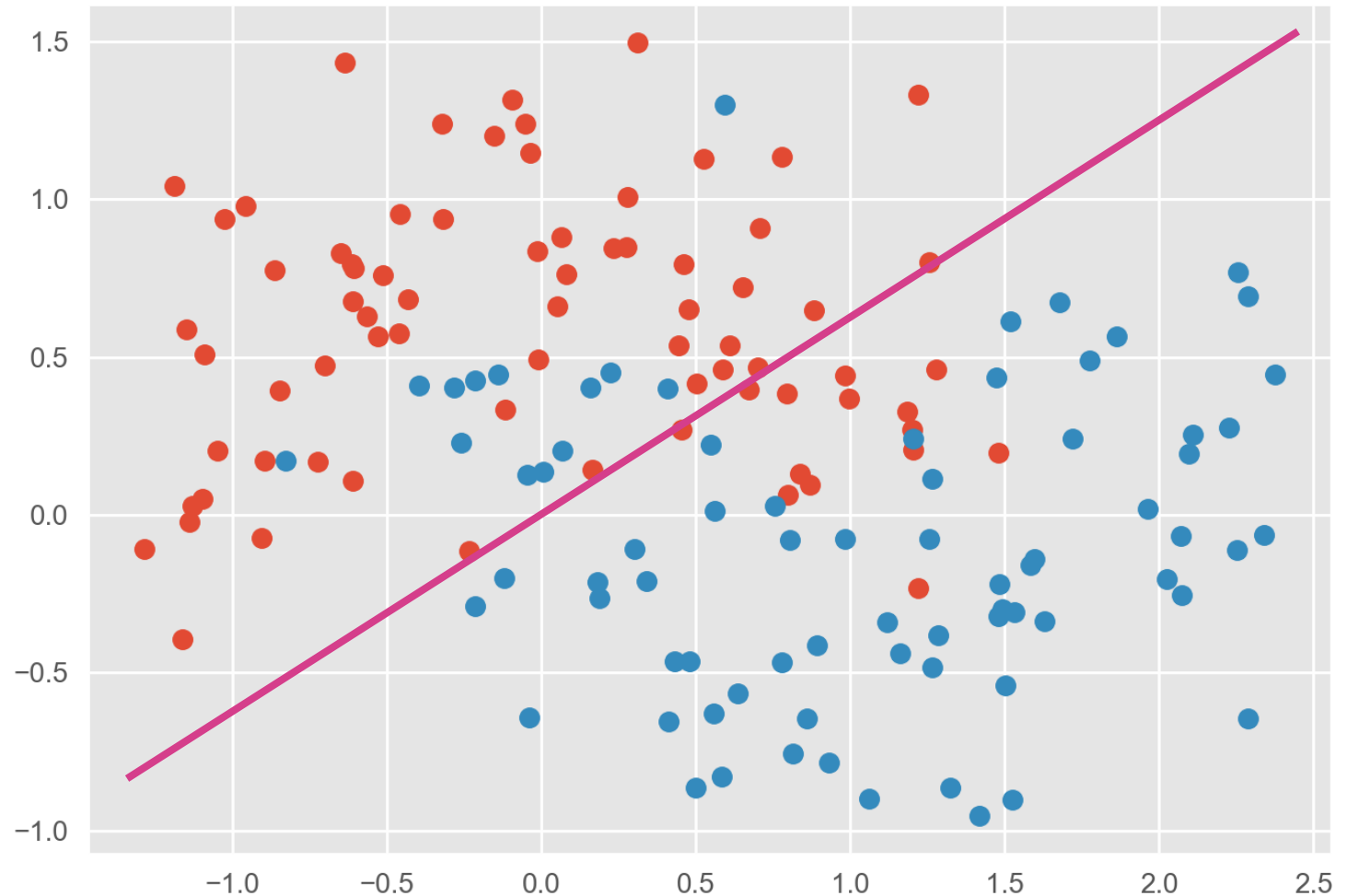
Logistic regression / GLMs can also overfit

Linear decision boundary

In this scenario ->
(# points $\gg$ # predictors)
it is not possible.

However, as # points
approaches # predictors,
we will see overfitting.

And for # points $\leq$ # predictors
(assuming linear
independence), we obtain (or
approach, for GLM) perfect
predictions.



Very brief review of overfitting in linear models

Suppose we have the following data

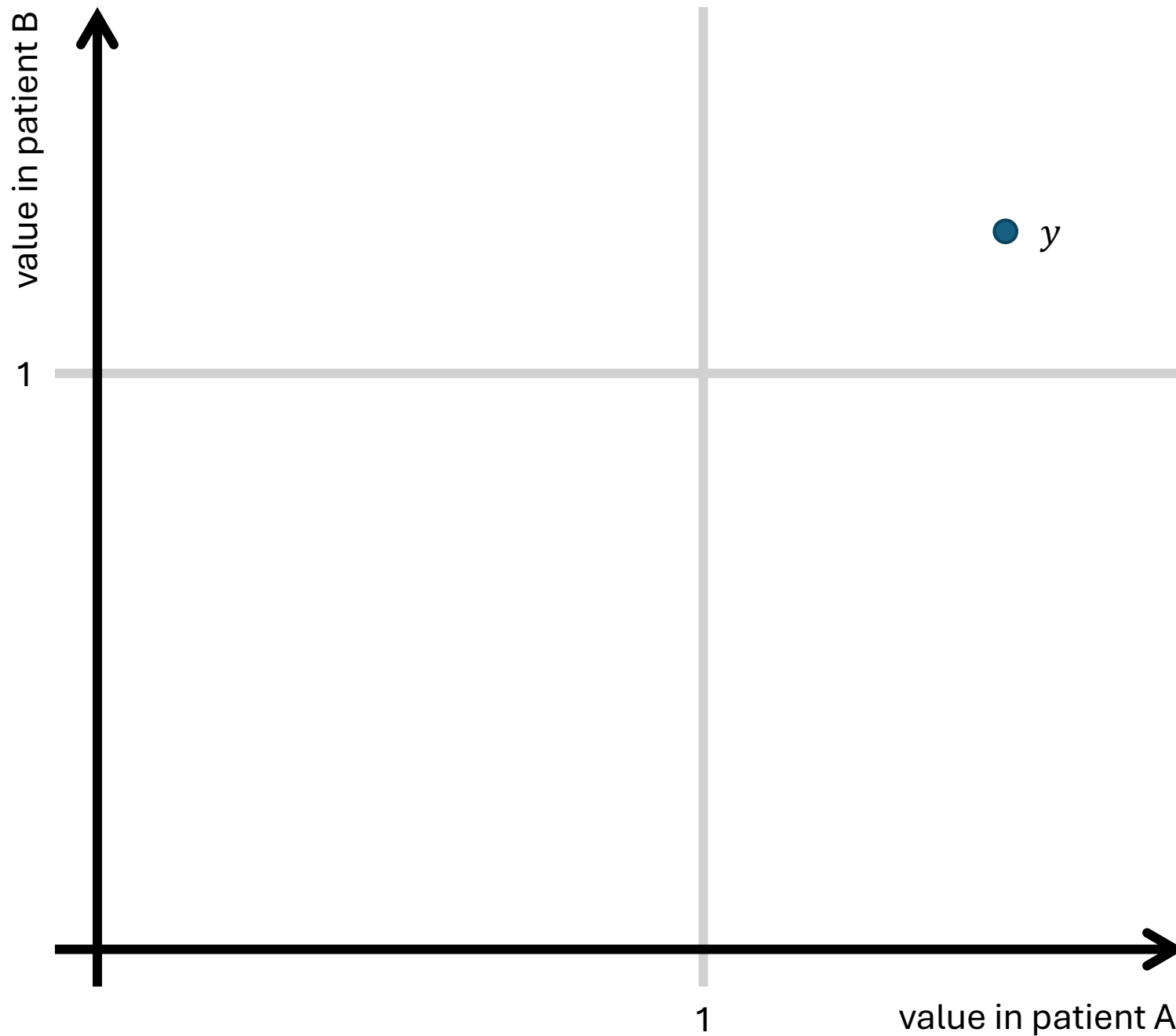
Patient	Predictor 1 (numeric)	Predictor 2 (numeric)	Outcome (numeric)
A	.5	.75	1.5
B	1	.75	1.25

> Goal: find the linear equation that best predicts the outcome

Patient	x_1	x_2	y
A	.5	.75	1.5
B	1	.75	1.25

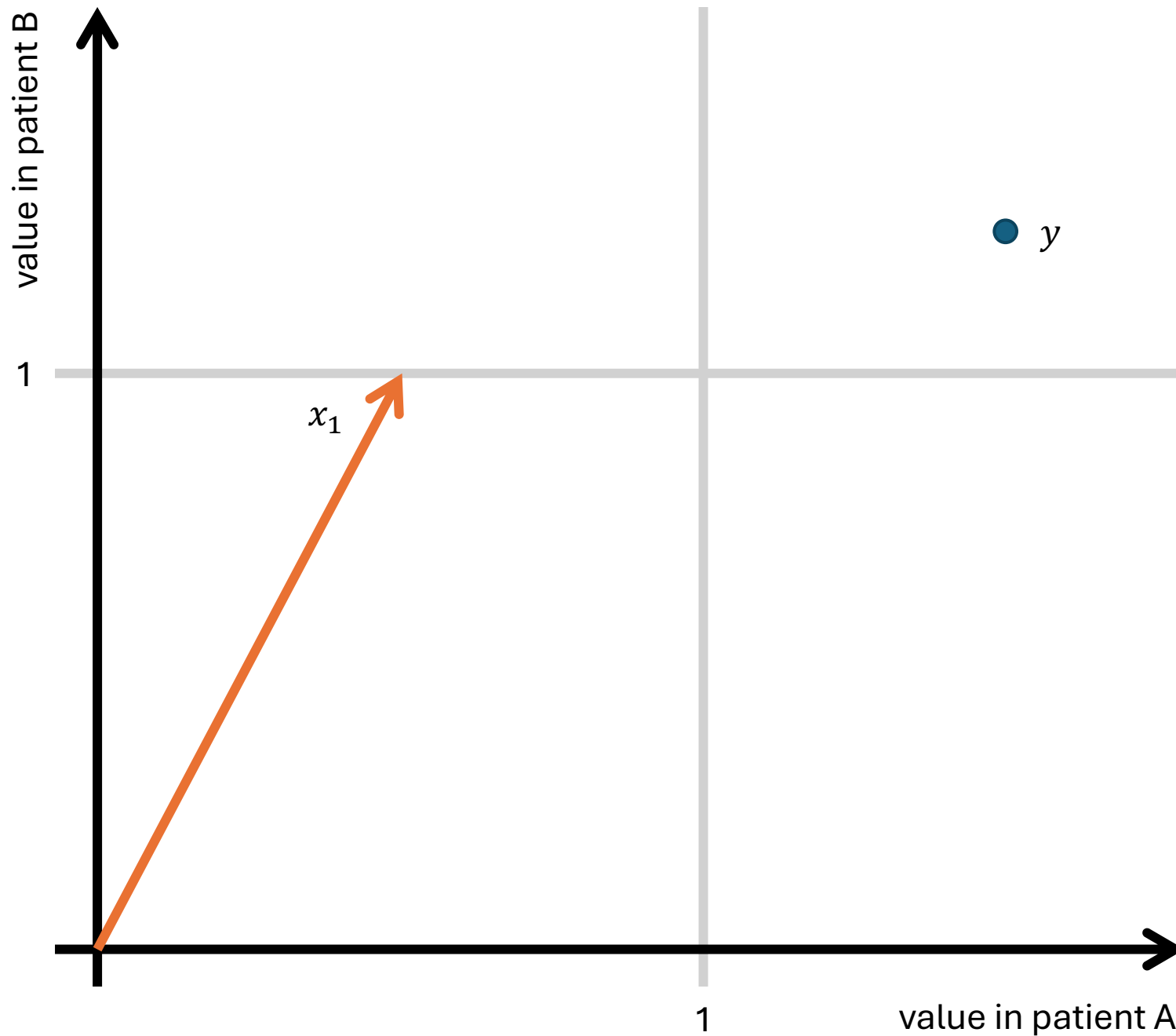
> Goal: find the linear equation that best predicts the outcome

$$b_1x_1 + b_2x_2 = y$$



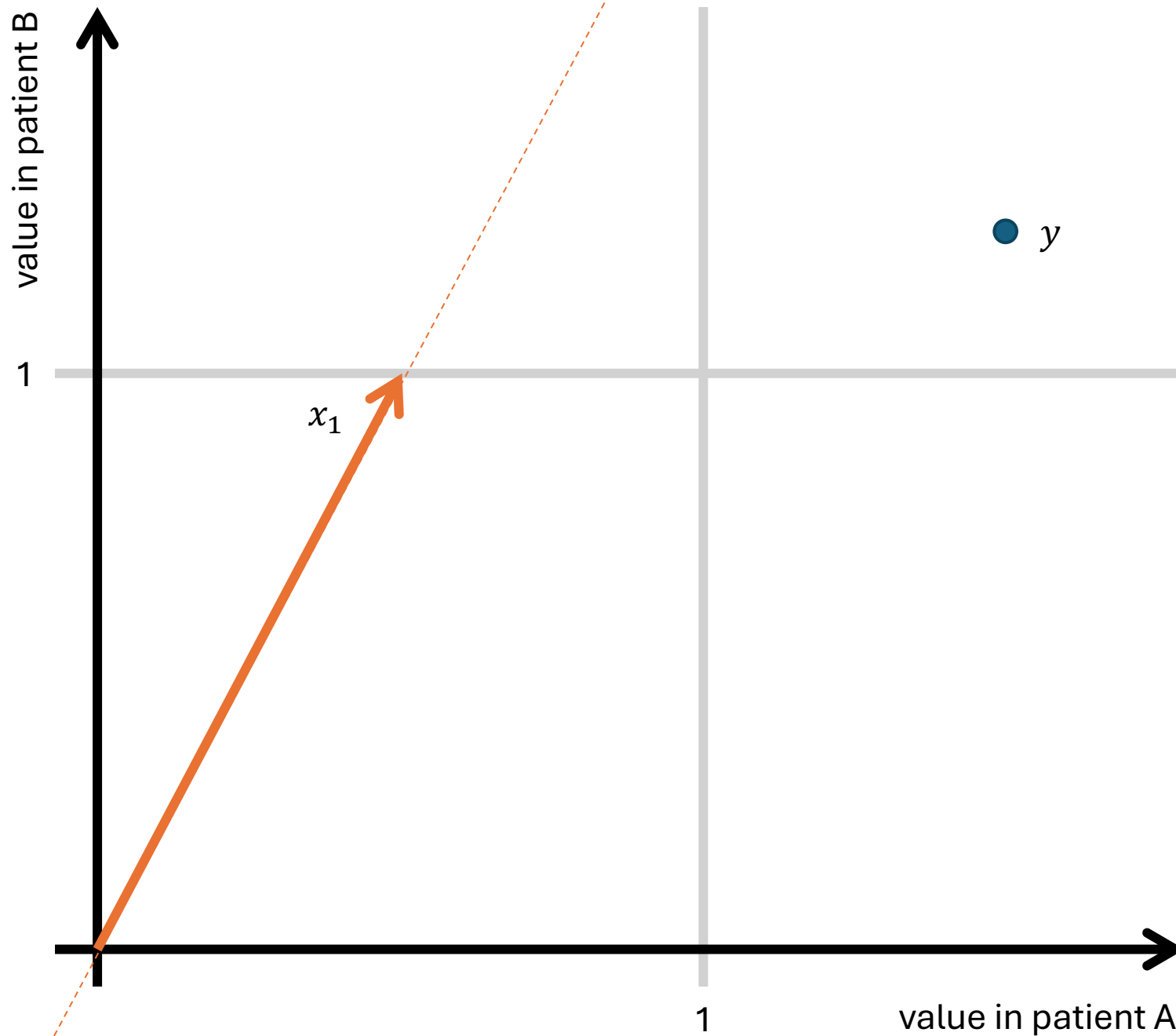
Short version:

- Outcomes define a point in N-dimensional space
- Each feature/predictor is a basis vector
- So, if we have at least N linearly independent features, we get perfect prediction
- Consider: are features linearly independent in practice?



Short version:

- Outcomes define a point in N-dimensional space
- Each feature/predictor is a basis vector
- So, if we have at least N linearly independent features, we get perfect prediction
- Consider: are features linearly independent in practice?

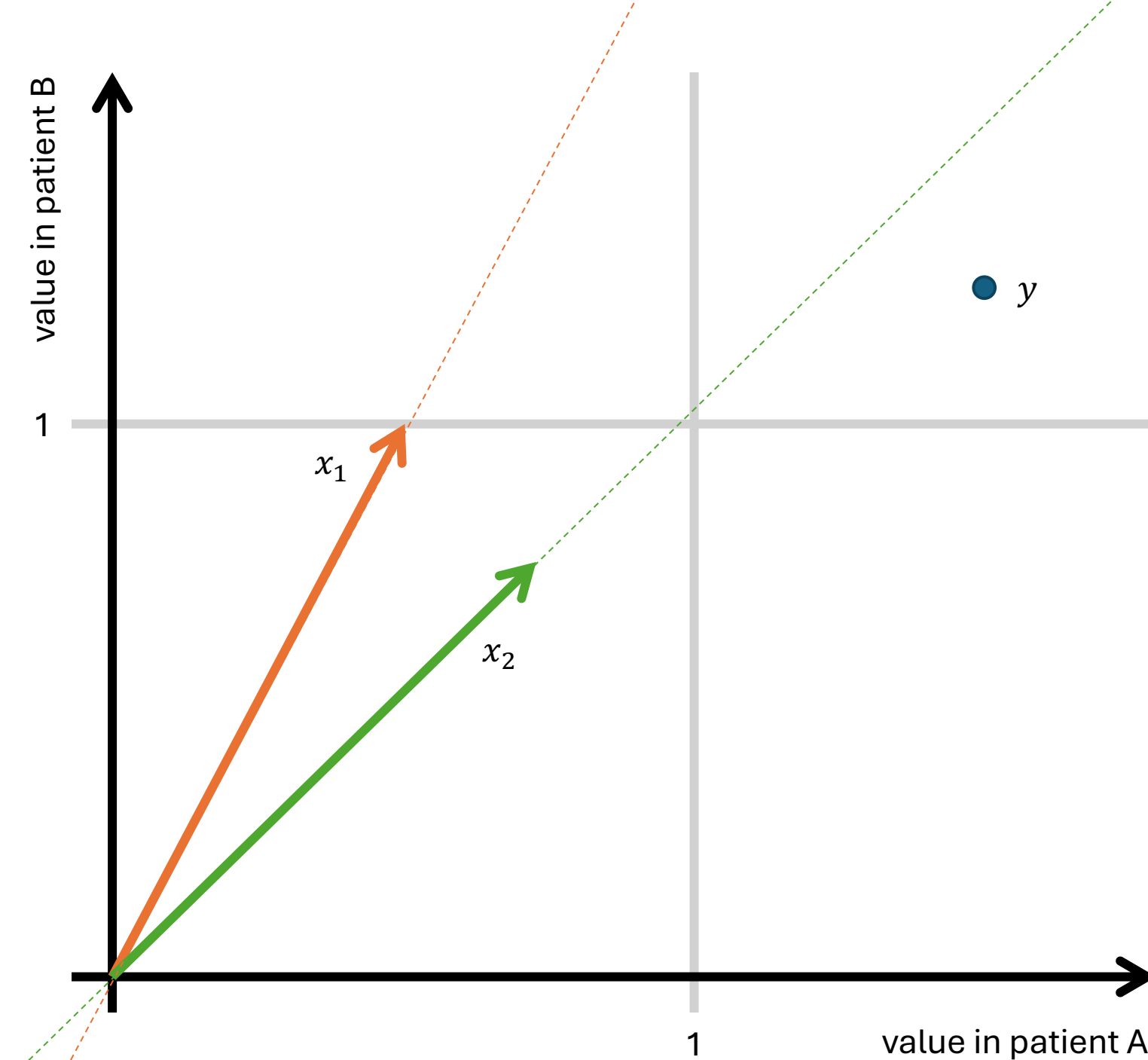


Goal: Predict y with x_1 only

From a graphical perspective, our goal is to get as close as possible to y , but we can only move in the x_1 direction

$$p < n$$

More patients than predictors.



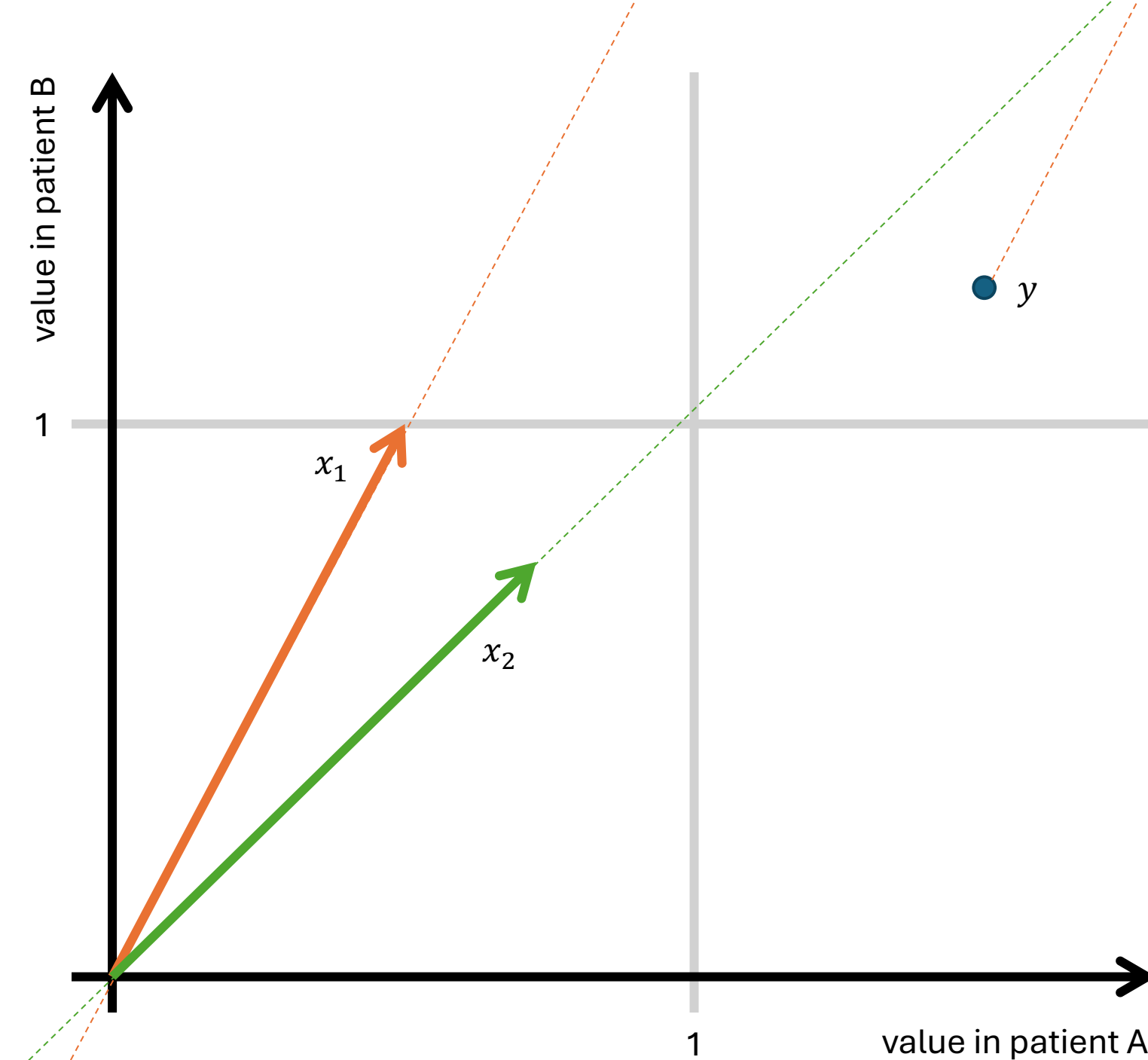
Goal: Predict y with x_1, x_2

From a graphical perspective, our goal is to get as close as possible to y . We can now move in both the x_1 direction and the x_2 direction.

$$p = n$$

Can always* predict perfectly on training set

*assuming linearly independent predictors



Goal: Predict y with x_1, x_2

From a graphical perspective, our goal is to get as close as possible to y . We can now move in both the x_1 direction and the x_2 direction.

$$p = n$$

Can always* predict perfectly on training set

*assuming linearly independent predictors

Takeaways

- When $\# \text{ points} \leq \# \text{ predictors}$, we can get perfect predictions (i.e. memorize the dataset)
- However, this typically requires very large model coefficients
- Regularization therefore prevents this behavior, and therefore tends to give us better generalization (i.e. out of sample) performance
- For logistic regression, “perfect” prediction requires predicting infinitely positive / negative log-odds, but we can get arbitrarily close to this using the same argument we used for linear regression

Regularization in PyTorch

```
import torch
import torch.nn as nn

# simple logistic regression
model = nn.Linear(d_in, 1)

criterion = nn.BCEWithLogitsLoss()
lambda_l2 = 1e-3

# forward pass
logits = model(X).squeeze()
data_loss = criterion(logits, y.float())

# manual L2 regularization (weights only, no bias)
l2_penalty = sum((p**2).sum() for name, p in
model.named_parameters()
                if "weight" in name)

loss = data_loss + lambda_l2 * l2_penalty
loss.backward()
```

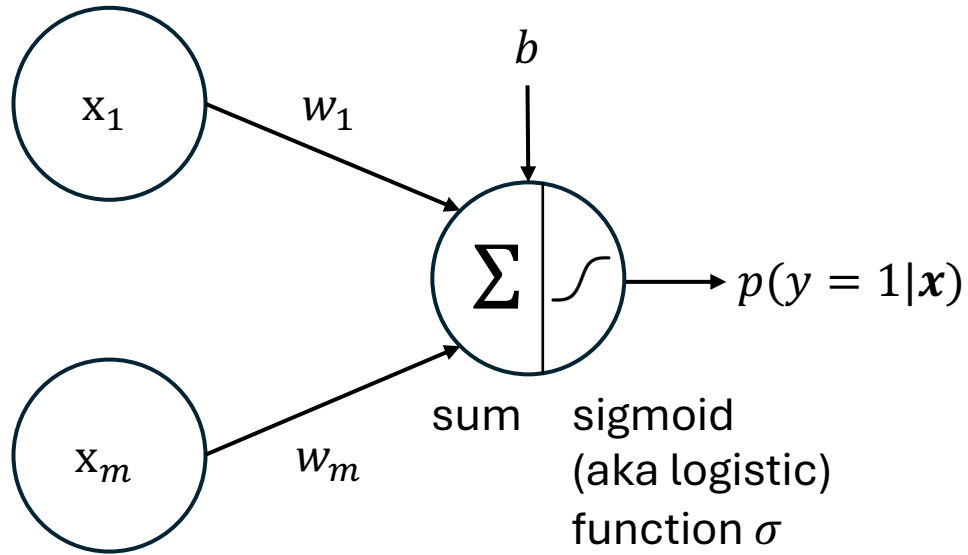
- Entirely flexible
- In ML we consider a wide variety of “regularizers”
- “Weight decay” is similar but not identical to L2 regularization

Calculating the gradient

Why autodifferentiation (i.e. backprop)?

- Numeric differentiation
 - Not exact
 - One forward pass *per parameter*
- Symbolic differentiation
 - Expressions become intractable
 - Numerically unstable
- Autodifferentiation
 - Exact
 - Requires only one forward pass and one backward pass through the network

Autodifferentiation for logistic regression



Write our model as $f_{\theta}(\mathbf{x})$, where $\theta = \{w_1, w_2, b\}$

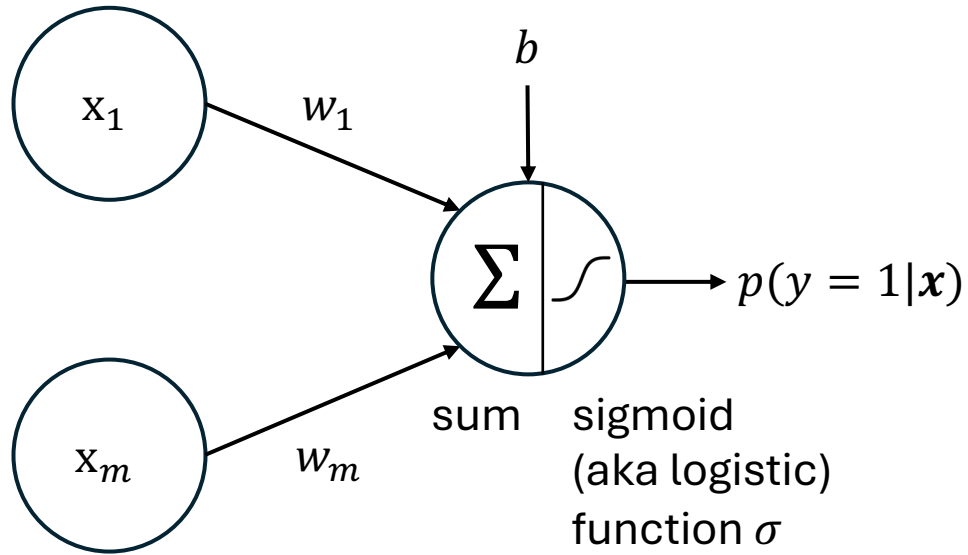
$$f_{\theta}(\mathbf{x}) = p = \sigma(w_1x_1 + w_2x_2 + b)$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$L(f_{\theta}(\mathbf{x}), y) = -\left(y \log f_{\theta}(\mathbf{x}) + (1 - y) \log (1 - f_{\theta}(\mathbf{x}))\right)$$

$$\text{Goal: obtain } \nabla_{\theta} L = \begin{pmatrix} \partial L / \partial w_1 \\ \partial L / \partial w_2 \\ \partial L / \partial b \end{pmatrix}$$

Autodifferentiation for logistic regression



Computational chain:

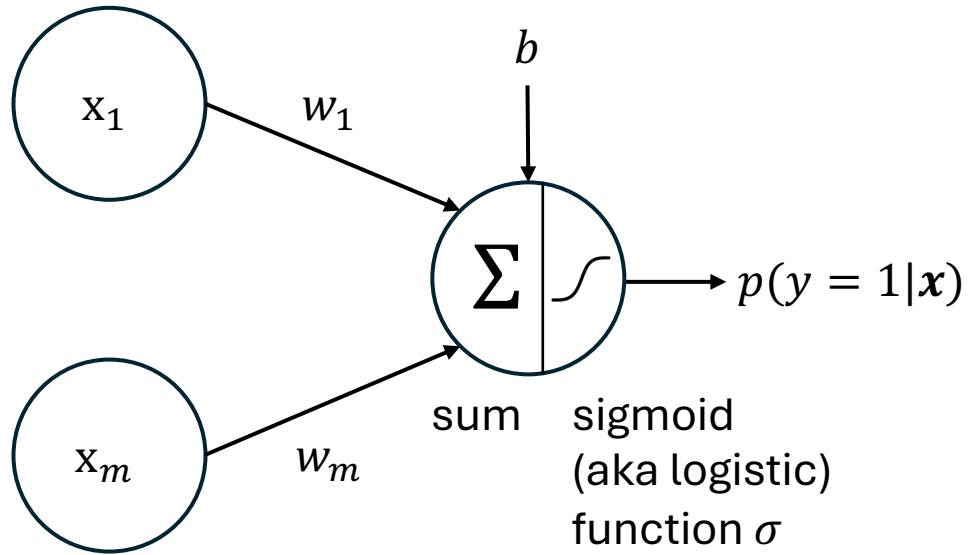
$$(w_1, w_2, b) \rightarrow z \rightarrow p \rightarrow L$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial p} * \frac{\partial p}{\partial z} * \frac{\partial z}{\partial w_1}$$

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial p} * \frac{\partial p}{\partial z} * \frac{\partial z}{\partial w_2}$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial p} * \frac{\partial p}{\partial z} * \frac{\partial z}{\partial b}$$

Autodifferentiation for logistic regression



Computational chain:

$$(w_1, w_2, b) \rightarrow z \rightarrow p \rightarrow L$$

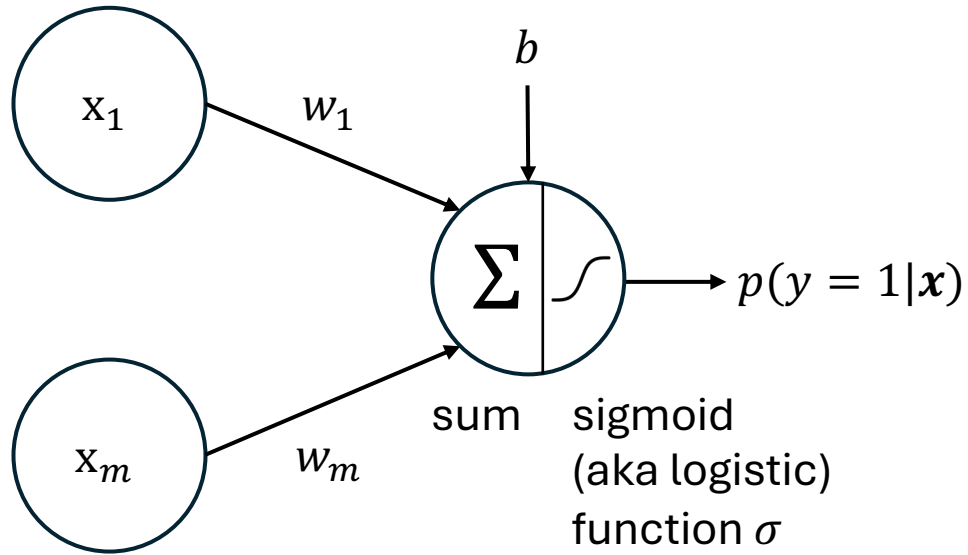
$$L(p) = -(y \log p + (1 - y) \log (1 - p))$$

$$\frac{\partial L}{\partial p} = -\left(\frac{y}{p} - \frac{1 - y}{1 - p}\right)$$

$$\frac{\partial p}{\partial z} = p * (1 - p)$$

$$\frac{\partial z}{\partial w_1} = x_1 \quad \frac{\partial z}{\partial w_2} = x_2 \quad \frac{\partial z}{\partial b} = 1$$

Autodifferentiation for logistic regression



Computational chain:

$$(w_1, w_2, b) \rightarrow z \rightarrow p \rightarrow L$$

$$L(p) = -(y \log p + (1 - y) \log (1 - p))$$

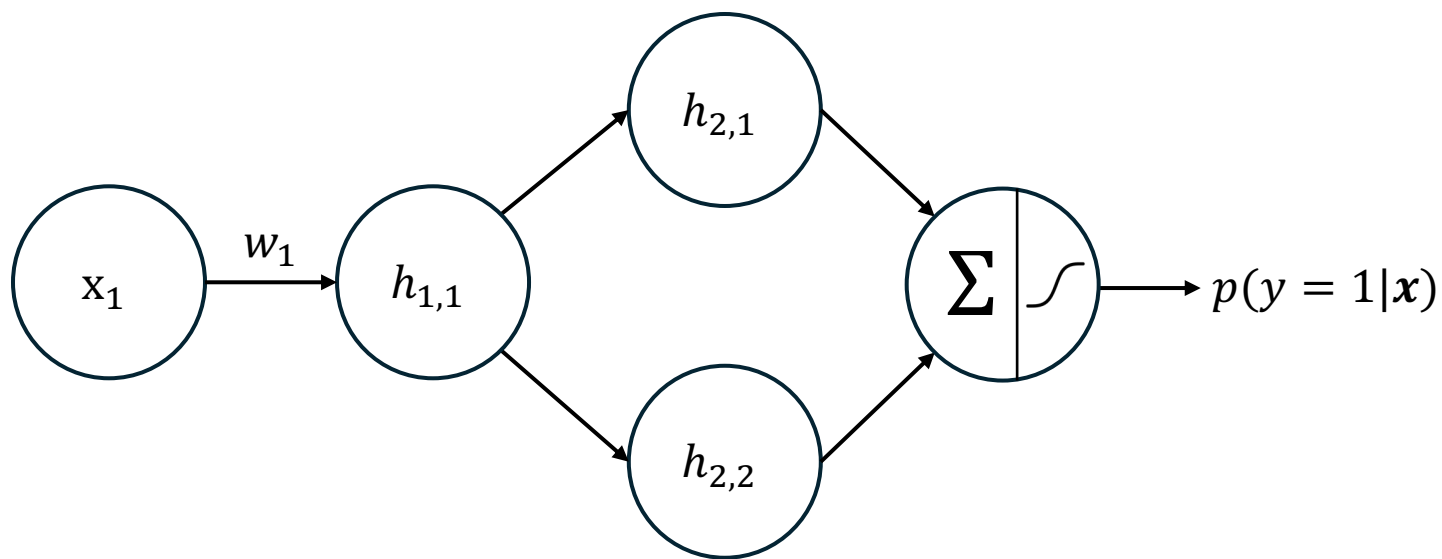
$$\nabla_{\theta} L = \begin{pmatrix} \partial L / \partial w_1 \\ \partial L / \partial w_2 \\ \partial L / \partial b \end{pmatrix} = (p - y) * \mathbf{x}$$

Autodiff in PyTorch

```
def bce_from_logits(logits, y):  
    return torch.mean(  
        torch.log(1 + torch.exp(logits)) - y * logits  
    )
```

- Each PyTorch primitive (e.g., `torch.log`) has a registered derivative
- These registered derivatives are composed in the backwards pass

Aggregating across multiple paths



Computational chain:

$$w_1 \rightarrow h_{1,1} \rightarrow (h_{2,1}, h_{2,2}) \rightarrow z \rightarrow p \rightarrow L$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial p} * \frac{\partial p}{\partial z} * \frac{\partial z}{\partial h_{2,1}} * \frac{\partial h_{2,1}}{\partial h_{1,1}} * \frac{\partial h_{1,1}}{\partial w_1} + \frac{\partial L}{\partial p} * \frac{\partial p}{\partial z} * \frac{\partial z}{\partial h_{2,2}} * \frac{\partial h_{2,2}}{\partial h_{1,1}} * \frac{\partial h_{1,1}}{\partial w_1}$$

Autodiff in PyTorch

```
def bce_from_logits(logits, y):  
    return torch.mean(  
        torch.log(1 + torch.exp(logits)) - y * logits  
    )
```

- Each PyTorch primitive (e.g., `torch.log`) has a registered derivative
- These registered derivatives are composed in the backwards pass
- Complex graphs require extensive bookkeeping, but the principles are simple

Parameter Optimization by **stochastic** gradient descent (SGD)

- Implemented by calculating exact gradient for a given minibatch
- Each minibatch yields a noisy estimate of the full-sample gradient
- Can be viewed as noise injection
- Injected noise helps us avoid local minima



Overview of common variations on SGD

Adaptation	Summary	Intuition	Example optimizer(s)
Momentum	Averages gradients over time	Roll a heavy ball downhill and push it based on the gradient	SGD with momentum, Adam
Adaptive learning rates	Different step sizes per parameter	Parameters with noisy gradients take smaller steps	RMSprop, Adam
Second-moment tracking	Tracks gradient variance	Commit to directions that keep changing	RMSprop, Adam
Weight decay correction	Ensures shrinkage is uniform across parameters	Shrinkage should not depend on gradient variance	SGD (no correction needed), AdamW
Bias correction	Fixes early-step underestimation	Warm up statistics before trusting them	Adam, AdamW

Activity: guess and check regression (i.e., **you** are the optimizer)

MORTALITY PREDICTION WORKSHEET												
COVARIATES							OUTCOMES AND PREDICTIONS					
patient	age	age_normalized	female	temp	temp_normalized		mortality	predicted_log_odds	predicted_prob	prediction	correct?	loss
0	30.5	-0.5	0	105.0	2.4		1	0.00	0.50	0	0	0.3010
1	74.0	1.1	1	96.7	-0.8		0	0.00	0.50	0	1	0.3010
2	27.4	-0.6	0	96.1	-1.0		0	0.00	0.50	0	1	0.3010
3	0.1	-1.5	1	98.5	-0.1		0	0.00	0.50	0	1	0.3010
4	0.7	-1.5	1	96.5	-0.9		0	0.00	0.50	0	1	0.3010
5	49.9	0.2	1	97.1	-0.6		0	0.00	0.50	0	1	0.3010
6	72.9	1.0	1	100.1	0.5		1	0.00	0.50	0	0	0.3010
7	29.1	-0.5	1	99.6	0.3		0	0.00	0.50	0	1	0.3010
8	83.5	1.4	1	100.6	0.7		1	0.00	0.50	0	0	0.3010
9	82.3	1.4	1	95.2	-1.3		1	0.00	0.50	0	0	0.3010
10	23.7	-0.7	0	99.4	0.2		1	0.00	0.50	0	0	0.3010
11	12.9	-1.1	0	96.6	-0.8		0	0.00	0.50	0	1	0.3010
12	53.9	0.4	1	100.3	0.6		0	0.00	0.50	0	1	0.3010
13	18.8	-0.9	0	98.6	0.0		0	0.00	0.50	0	1	0.3010
14	51.8	0.3	0	98.5	-0.1		0	0.00	0.50	0	1	0.3010
15	3.3	-1.4	0	94.6	-1.6		0	0.00	0.50	0	1	0.3010
16	69.7	0.9	0	99.1	0.1		0	0.00	0.50	0	1	0.3010
17	60.4	0.6	1	104.2	2.1		1	0.00	0.50	0	0	0.3010
18	73.6	1.1	1	99.1	0.1		1	0.00	0.50	0	0	0.3010
19	53.3	0.3	1	99.1	0.1		0	0.00	0.50	0	1	0.3010
PARAMETERS							PERFORMANCE					
		b_age	b_female		b_temp	bias					accuracy	avg_loss
	guess	0.00	0.00		0.00	0.00					0.65	0.3010
	optimal											